Linux Web Server

Name: Dharani Thakkallapally
Student ID: 16354872

Setup Linux server

Step 1: Start Apache server.

```
dharanithakkallapally@Dharanis-MacBook-Air ~ % sudo apachectl start

Password:
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using Dharanis-MacBook-Air.local. Set the 'ServerName' directive globally to suppress this message
httpd (pid 78176) already running
dharanithakkallapally@Dharanis-MacBook-Air ~ %
```

Step 2: Installed Homebrew in mac terminal.

```
Last login: Mon Nov 25 09:20:48 on ttys000
dharanithakkallapally@Dharanis-MacBook-Air ~ % /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
==> Checking for `sudo` access (which may request your password)...
Password:
==> This script will install:
/opt/homebrew/bin/brew
/opt/homebrew/share/doc/homebrew
/opt/homebrew/share/man/man1/brew.1
/opt/homebrew/share/zsh/site-functions/_brew
/opt/homebrew/etc/bash_completion.d/brew
/opt/homebrew

Press RETURN/ENTER to continue or any other key to abort:
==> /usr/bin/sudo /usr/sbin/chown -R dharanithakkallapally:admin /opt/homebrew
==> Downloading and installing Homebrew...
remote: Enumerating objects: 204, done.
remote: Counting objects: 100% (174/174), done.
remote: Compressing objects: 100% (68/68), done.
remote: Total 204 (delta 107), reused 152 (delta 99), pack-reused 30 (from 1)
==> Updating Homebrew...
Updated 2 taps (homebrew/core and homebrew/cask).
==> Installation successful!

==> Homebrew has enabled anonymous aggregate formulae and cask analytics.
Read the analytics documentation (and how to opt-out) here:
  https://docs.brew.sh/Analytics
No analytics data has been sent yet (nor will any be during this install run).

==> Homebrew is run entirely by unpaid volunteers. Please consider donating:
  https://github.com/Homebrew/brew#donations

==> Next steps:
- Run brew help to get started
- Further documentation:
    https://docs.brew.sh
```

Step 3: Install MySQL using brew.

```
dharanithakkallapally@Dharanis-MacBook-Air ~ % brew install mysql

==> Downloading https://formulae.brew.sh/api/formula.jws.json
######################################################################################################################################################################################################################################## 100.0%
==> Downloading https://formulae.brew.sh/api/cask.jws.json
######################################################################################################################################################################################################################################## 100.0%
Warning: mysql 9.0.1_6 is already installed and up-to-date.
To reinstall 9.0.1_6, run:
  brew reinstall mysql
dharanithakkallapally@Dharanis-MacBook-Air ~ %
```

Step 4: Start MySQL services using brew.

```
dharanithakkallapally@Dharanis-MacBook-Air ~ % brew services start mysql

==> Successfully started `mysql` (label: homebrew.mxcl.mysql)
dharanithakkallapally@Dharanis-MacBook-Air ~ % mysql_secure_installation


Securing the MySQL server deployment.

Enter password for user root:
Error: Access denied for user 'root'@'localhost' (using password: YES)
dharanithakkallapally@Dharanis-MacBook-Air ~ % mysql -u root -p

Enter password:
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: YES)
dharanithakkallapally@Dharanis-MacBook-Air ~ %
```

Step 5: Secure MySQL installation by setting up password.

```
dharanithakkallapally@Dharanis-MacBook-Air ~ % mysql_secure_installation


Securing the MySQL server deployment.

Connecting to MySQL using a blank password.
The 'validate_password' component is installed on the server.
The subsequent steps will run with the existing configuration
of the component.
Please set the password for root here.

New password:

Re-enter new password:
Sorry, passwords do not match.

New password:

Re-enter new password:

Estimated strength of the password: 50
Do you wish to continue with the password provided?(Press y|Y for Yes, any other key for No) : y
 ... Failed! Error: Your password does not satisfy the current policy requirements

New password:

Re-enter new password:

Estimated strength of the password: 100
Do you wish to continue with the password provided?(Press y|Y for Yes, any other key for No) : y
By default, a MySQL installation has an anonymous user,
allowing anyone to log into MySQL without having to have
a user account created for them. This is intended only for
testing, and to make the installation go a bit smoother.
You should remove them before moving into a production
environment.

Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.


Normally, root should only be allowed to connect from
'localhost'. This ensures that someone cannot guess at
the root password from the network.

Disallow root login remotely? (Press y|Y for Yes, any other key for No) : n

 ... skipping.
By default, MySQL comes with a database named 'test' that
anyone can access. This is also intended only for testing,
and should be removed before moving into a production
environment.


Remove test database and access to it? (Press y|Y for Yes, any other key for No) : n

 ... skipping.
Reloading the privilege tables will ensure that all changes
made so far will take effect immediately.

Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.

All done!
dharanithakkallapally@Dharanis-MacBook-Air ~ %
```

Step 6: Login to MySQL using root user account.

```
dharanithakkallapally@Dharanis-MacBook-Air ~ % mysql -u root -p

Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 9.0.1 Homebrew

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Step 7: Create tables for users and files using below SQL commands.

```
CREATE DATABASE nas_db;
USE nas_db;

CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   username VARCHAR(50),
   password VARCHAR(255),
   role ENUM('admin', 'user'),
   permissions JSON
);

CREATE TABLE files (
   id INT AUTO_INCREMENT PRIMARY KEY,
   file_name VARCHAR(255),
   file_path VARCHAR(255),
   uploaded_by VARCHAR(50),
   upload_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
dharanithakkallapally@Dharanis-MacBook-Air ~ % mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 38
Server version: 9.0.1 Homebrew

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
mysql> CREATE DATABASE nas_db;
ERROR 1007 (HY000): Can't create database 'nas_db'; database exists
mysql> USE nas_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
mysql> CREATE TABLE users (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     username VARCHAR(50),
    ->     password VARCHAR(255),
    ->     role ENUM('admin', 'user'),
    ->     permissions JSON
    -> );
ERROR 1050 (42S01): Table 'users' already exists
mysql>
mysql> CREATE TABLE files (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     file_name VARCHAR(255),
    ->     file_path VARCHAR(255),
    ->     uploaded_by VARCHAR(50),
    ->     upload_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
ERROR 1050 (42S01): Table 'files' already exists
mysql>
```

Step 8: Exit MySQL

```
mysql> exit
Bye
```

Step 9: Install Python.

```
dharanithakkallapally@Dharanis-MacBook-Air ~ % brew install python

==> Downloading https://formulae.brew.sh/api/formula.jws.json
######################################################################################################################################################### 100.0%
==> Downloading https://formulae.brew.sh/api/cask.jws.json
######################################################################################################################################################### 100.0%
Warning: python@3.13 3.13.0_1 is already installed and up-to-date.
To reinstall 3.13.0_1, run:
  brew reinstall python@3.13
dharanithakkallapally@Dharanis-MacBook-Air ~ % pip3 install flask pymysql


[notice] A new release of pip is available: 24.2 -> 24.3.1
[notice] To update, run: python3.13 -m pip install --upgrade pip
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try brew install
    xyz, where xyz is the package you are trying to
    install.

    If you wish to install a Python library that isn't in Homebrew,
    use a virtual environment:

    python3 -m venv path/to/venv
    source path/to/venv/bin/activate
    python3 -m pip install xyz

    If you wish to install a Python application that isn't in Homebrew,
    it may be easiest to use 'pipx install xyz', which will manage a
    virtual environment for you. You can install pipx with

    brew install pipx

    You may restore the old behavior of pip by passing
    the '--break-system-packages' flag to pip, or by adding
    'break-system-packages = true' to your pip.conf file. The latter
    will permanently disable this error.

    If you disable this error, we STRONGLY recommend that you additionally
    pass the '--user' flag to pip, or set 'user = true' in your pip.conf
    file. Failure to do this can result in a broken Homebrew installation.

    Read more about this behavior here: <https://peps.python.org/pep-0668/>

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python installation or OS, by passing --break-s
ystem-packages.
hint: See PEP 668 for the detailed specification.
dharanithakkallapally@Dharanis-MacBook-Air ~ %
```

Step 10: Create a project folder using mkdir command and change it as present working directory. Then create a virtual environment to install flask and PyMySQL inside the directory.

```
dharanithakkallapally@Dharanis-MacBook-Air ~ % touch index.html

dharanithakkallapally@Dharanis-MacBook-Air ~ %
```

```
dharanithakkallapally@Dharanis-MacBook-Air nas_project % python3 -m venv flask_venv
dharanithakkallapally@Dharanis-MacBook-Air nas_project % ls -ltr
total 16
-rw-r--r--@ 1 dharanithakkallapally  staff  342 Nov 27 09:56 f.html
-rw-r--r--@ 1 dharanithakkallapally  staff  844 Nov 27 09:57 b.py
drwxr-xr-x  7 dharanithakkallapally  staff  224 Nov 27 10:24 flask_venv
dharanithakkallapally@Dharanis-MacBook-Air nas_project % . flask_venv/bin/activate
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project % pip install flask
Collecting flask
  Downloading flask-3.1.0-py3-none-any.whl.metadata (2.7 kB)
Collecting Werkzeug>=3.1 (from flask)
  Downloading werkzeug-3.1.3-py3-none-any.whl.metadata (3.7 kB)
Collecting Jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.4-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.1.7-py3-none-any.whl.metadata (3.0 kB)
Collecting blinker>=1.9 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting MarkupSafe>=2.0 (from Jinja2>=3.1.2->flask)
  Downloading MarkupSafe-3.0.2-cp313-cp313-macosx_11_0_arm64.whl.metadata (4.0 kB)
Downloading flask-3.1.0-py3-none-any.whl (102 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.1.7-py3-none-any.whl (97 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.4-py3-none-any.whl (133 kB)
Downloading werkzeug-3.1.3-py3-none-any.whl (224 kB)
Downloading MarkupSafe-3.0.2-cp313-cp313-macosx_11_0_arm64.whl (12 kB)
Installing collected packages: MarkupSafe, itsdangerous, click, blinker, Werkzeug, Jinja2, flask
Successfully installed Jinja2-3.1.4 MarkupSafe-3.0.2 Werkzeug-3.1.3 blinker-1.9.0 click-8.1.7 flask-3.1.0 itsdangerous-2.2.0

[notice] A new release of pip is available: 24.2 -> 24.3.1
[notice] To update, run: pip install --upgrade pip
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project % python3 b.py
Traceback (most recent call last):
  File "/Users/dharanithakkallapally/nas_project/b.py", line 3, in <module>
    import pymysql
ModuleNotFoundError: No module named 'pymysql'
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project % pip install pymysql

Collecting pymysql
  Downloading PyMySQL-1.1.1-py3-none-any.whl.metadata (4.4 kB)
Downloading PyMySQL-1.1.1-py3-none-any.whl (44 kB)
Installing collected packages: pymysql
Successfully installed pymysql-1.1.1

[notice] A new release of pip is available: 24.2 -> 24.3.1
[notice] To update, run: pip install --upgrade pip
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project % python3 b.py
Traceback (most recent call last):
  File "/Users/dharanithakkallapally/nas_project/b.py", line 8, in <module>
    db = pymysql.connect(host='localhost', user='root', password='your_password', database='nas_db')
```

Develop Web interface

Step 11: Create a html file using touch command. Open that html file in vs code editor. Enter the html code and save the script file. Place the html file inside templates folder of project directory.

```
dharanithakkallapally@Dharanis-MacBook-Air nas_project % touch f.html
dharanithakkallapally@Dharanis-MacBook-Air nas_project %
```

Step 12: Now create py file using touch command to perform backend operation. Open that py file in vs code editor. Enter the python code and save the script file.

```
dharanithakkallapally@Dharanis-MacBook-Air nas_project % touch b.py
dharanithakkallapally@Dharanis-MacBook-Air nas_project %
```

Step 13: Activate the flask virtual environment.

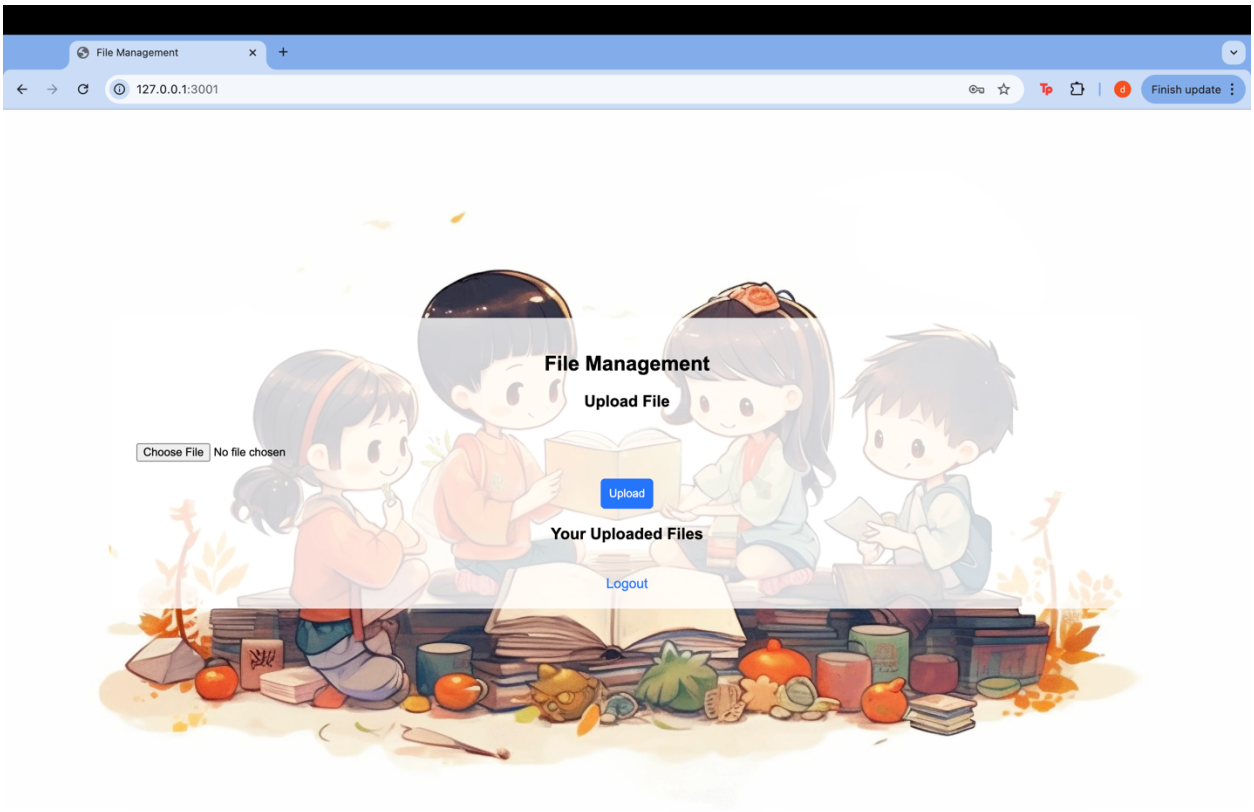
```
dharanithakkallapally@Dharanis-MacBook-Air nas_project % . flask_venv/bin/activate
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project %
```

Step 14: Run the .py file

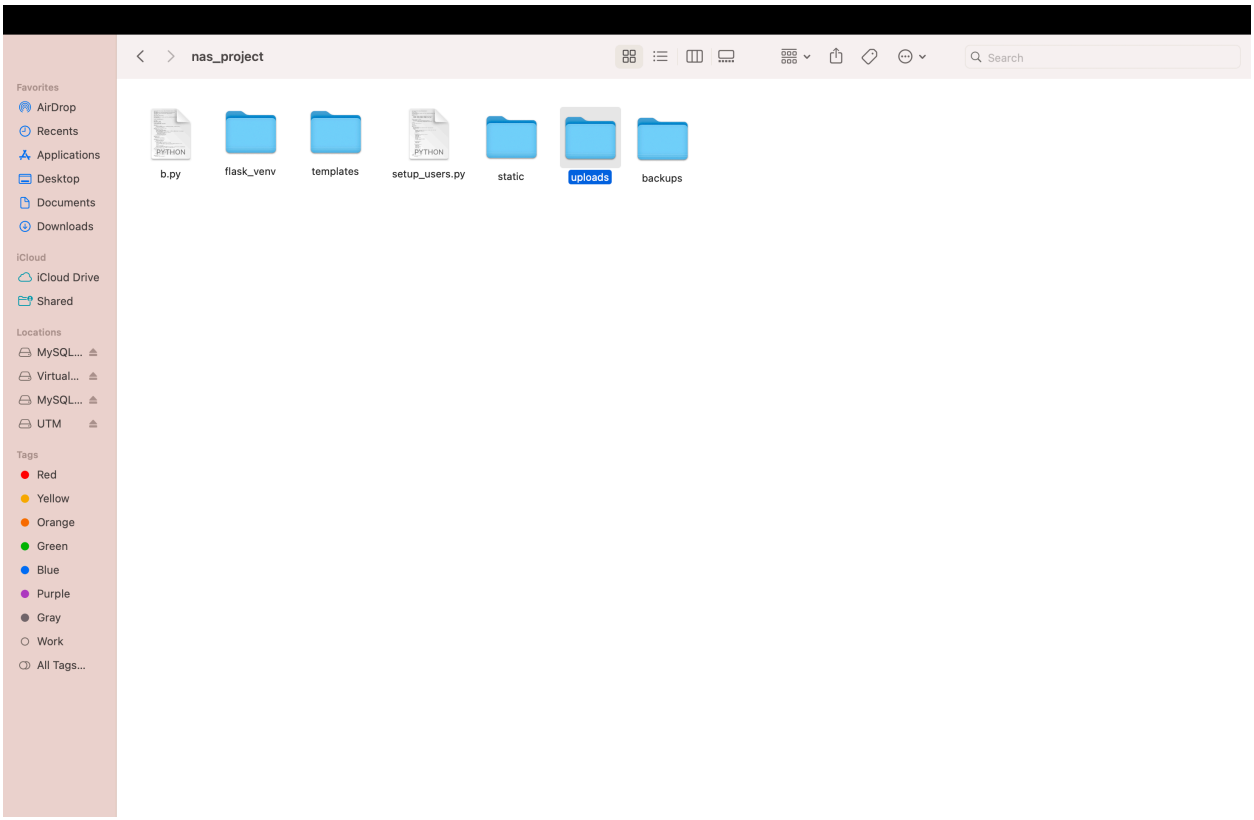
```
dharanithakkallapally@Dharanis-MacBook-Air nas_project % . flask_venv/bin/activate
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project % python3 b.py
 * Serving Flask app 'b'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:3001
 * Running on http://192.168.1.137:3001
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 104-655-575
```

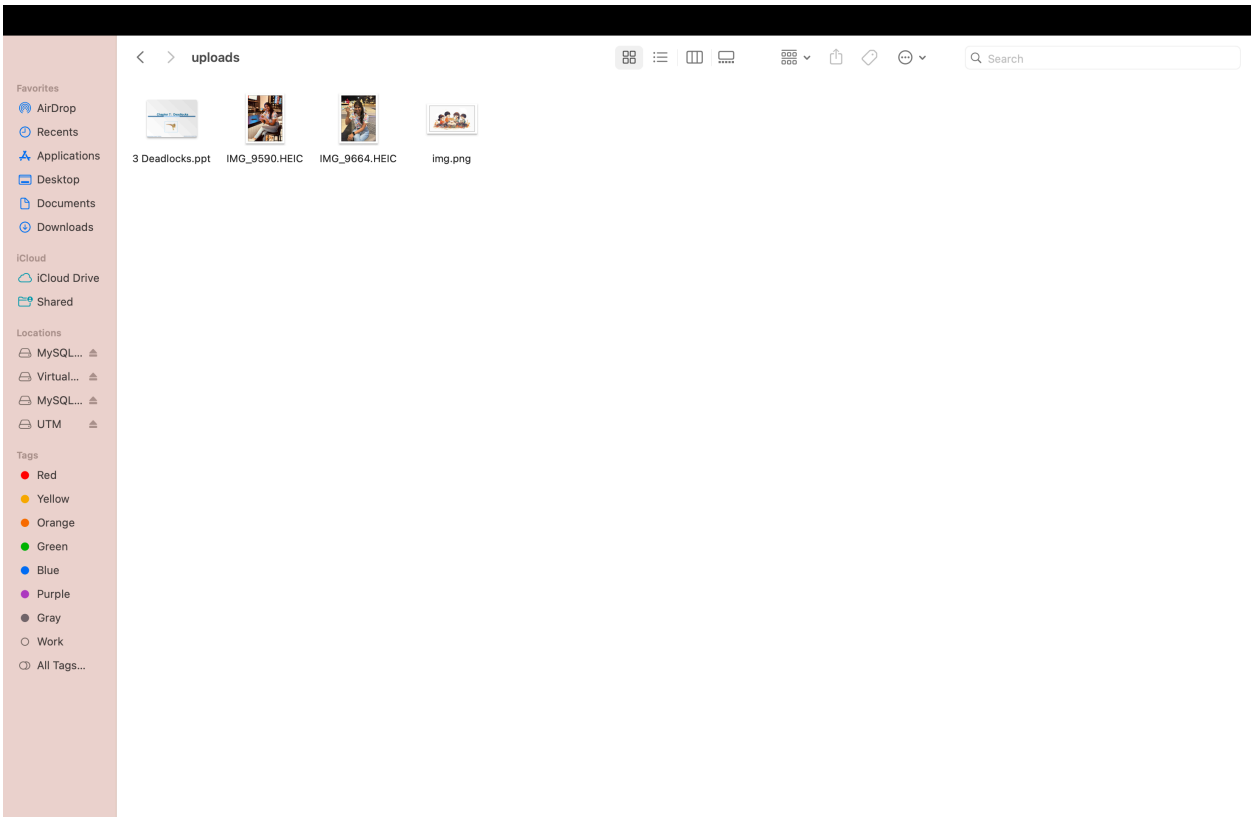
Step 15: Try to access the webserver using the Ip address u have after running the py file.

```
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project % python3 b.py
 * Serving Flask app 'b'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:3001
 * Running on http://192.168.1.137:3001
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 104-655-575
127.0.0.1 - - [28/Nov/2024 13:17:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [28/Nov/2024 13:18:31] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [28/Nov/2024 13:32:08] "GET / HTTP/1.1" 200 -
 * Detected change in '/Users/dharanithakkallapally/nas_project/b.py', reloading
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 104-655-575
 * Detected change in '/Users/dharanithakkallapally/nas_project/b.py', reloading
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 104-655-575
 * Detected change in '/Users/dharanithakkallapally/nas_project/b.py', reloading
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 104-655-575
```



Step 16: Any uploaded file will be organized into a separate folder within the designated project directory.


nas_project
Search
Favorites
AirDrop
Recents
Applications
Desktop
Documents
Downloads
iCloud
iCloud Drive
Shared
Locations
MySQL...
Virtual...
MySQL...
UTM
Tags
Red
Yellow
Orange
Green
Blue
Purple
Gray
Work
All Tags...
b.py
flask_venv
templates
setup_users.py
static
uploads
backups


uploads
Search
Favorites
AirDrop
Recents
Applications
Desktop
Documents
Downloads
iCloud
iCloud Drive
Shared
Locations
MySQL...
Virtual...
MySQL...
UTM
Tags
Red
Yellow
Orange
Green
Blue
Purple
Gray
Work
All Tags...
3 Deadlocks.ppt
IMG_9590.HEIC
IMG_9664.HEIC
img.png

Configure port forwarding

Step 17: Perform port forwarding by opening routers IP and logging in using admin credentials. Click on port forwarding settings and add new rule with details service name, internal IP, Internal port, External port, protocol. Restart router. Tested the public IP.

Implement user access control

Step 18: Create an extra script file to setup a set of users that can login the NAS webserver. Save the user details py file in. project folder.

Step 19: Implement user access control by creating a html for user logins. Also Modify the backend code accordingly. Place login.html file in templates folder.

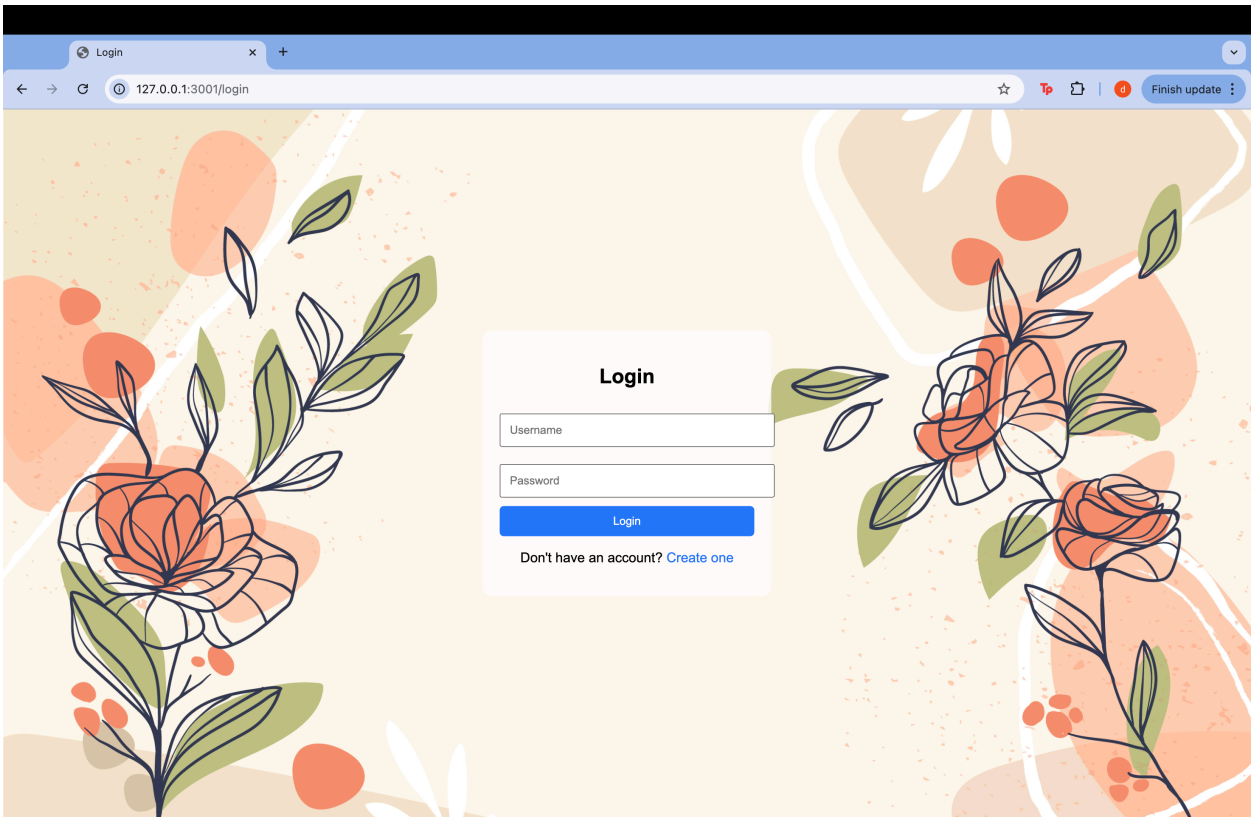
Step 20: Run setup_users py file.

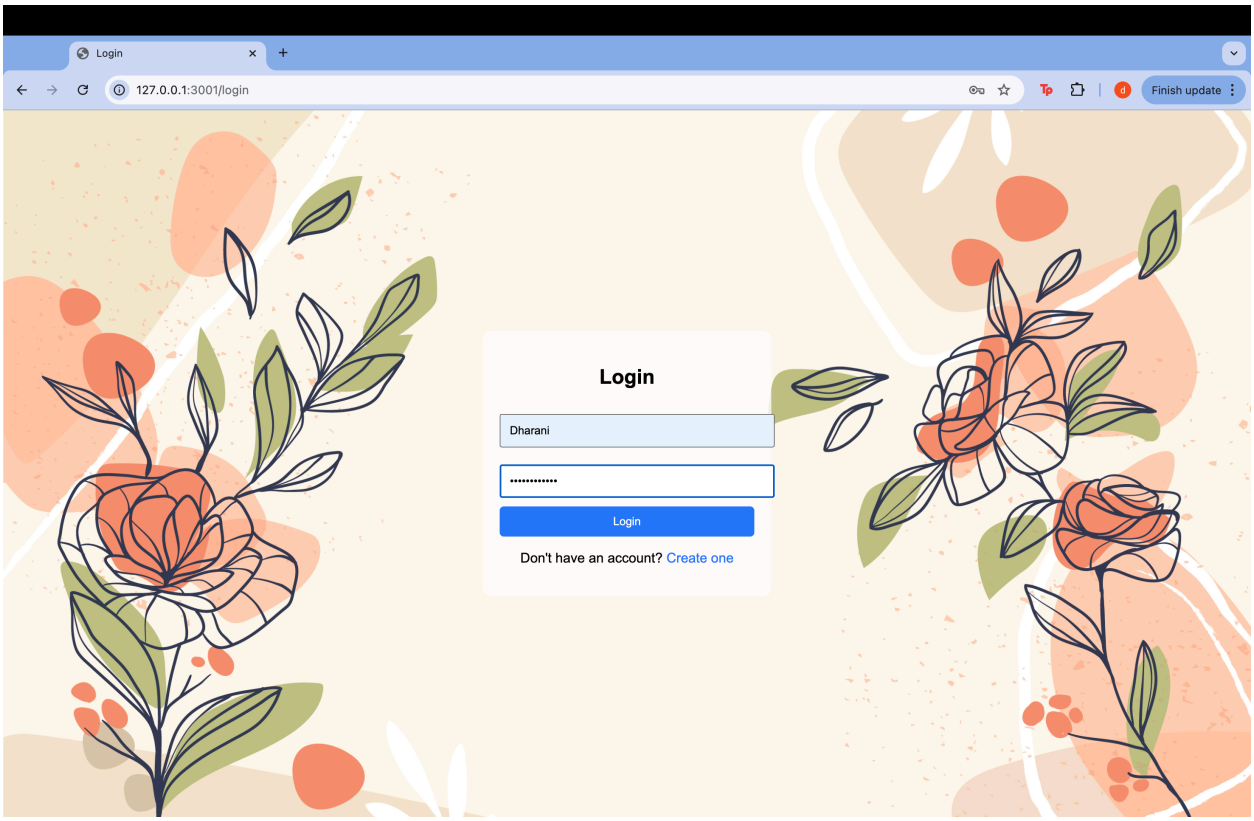
```
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project % python3 setup_users.py
Users added successfully.
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project %
```

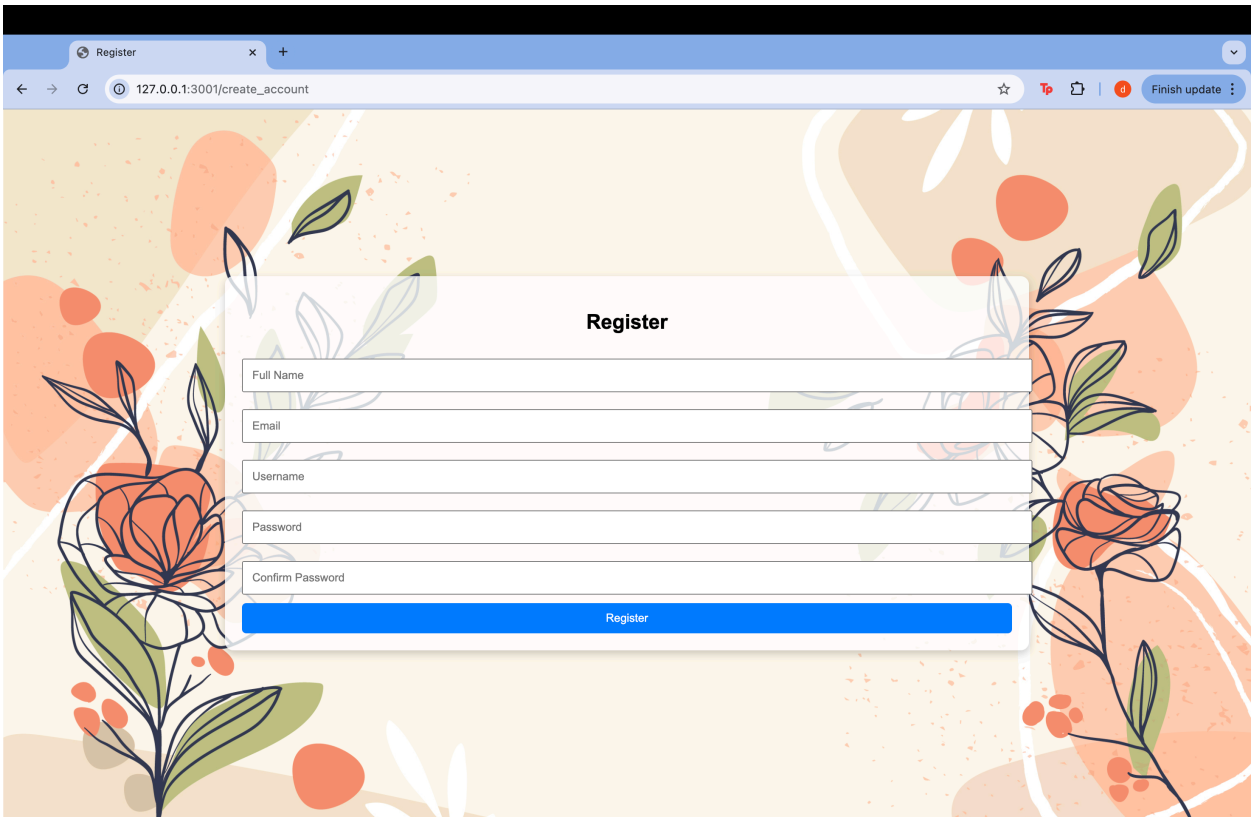
Step 21: Run the backend py file.

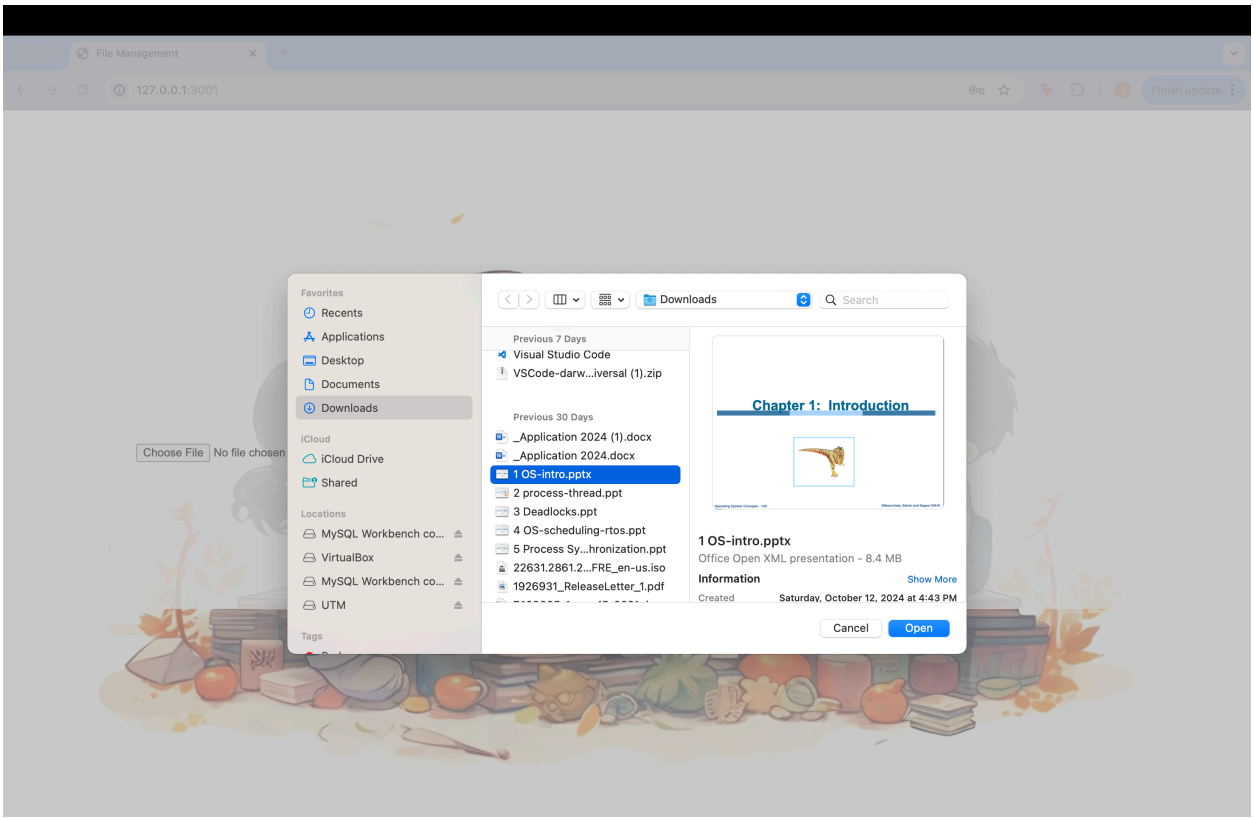
```
(flask_venv) dharanithakkallapally@Dharanis-MacBook-Air nas_project % python3 b.py
 * Serving Flask app 'b'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:3001
 * Running on http://192.168.1.137:3001
```

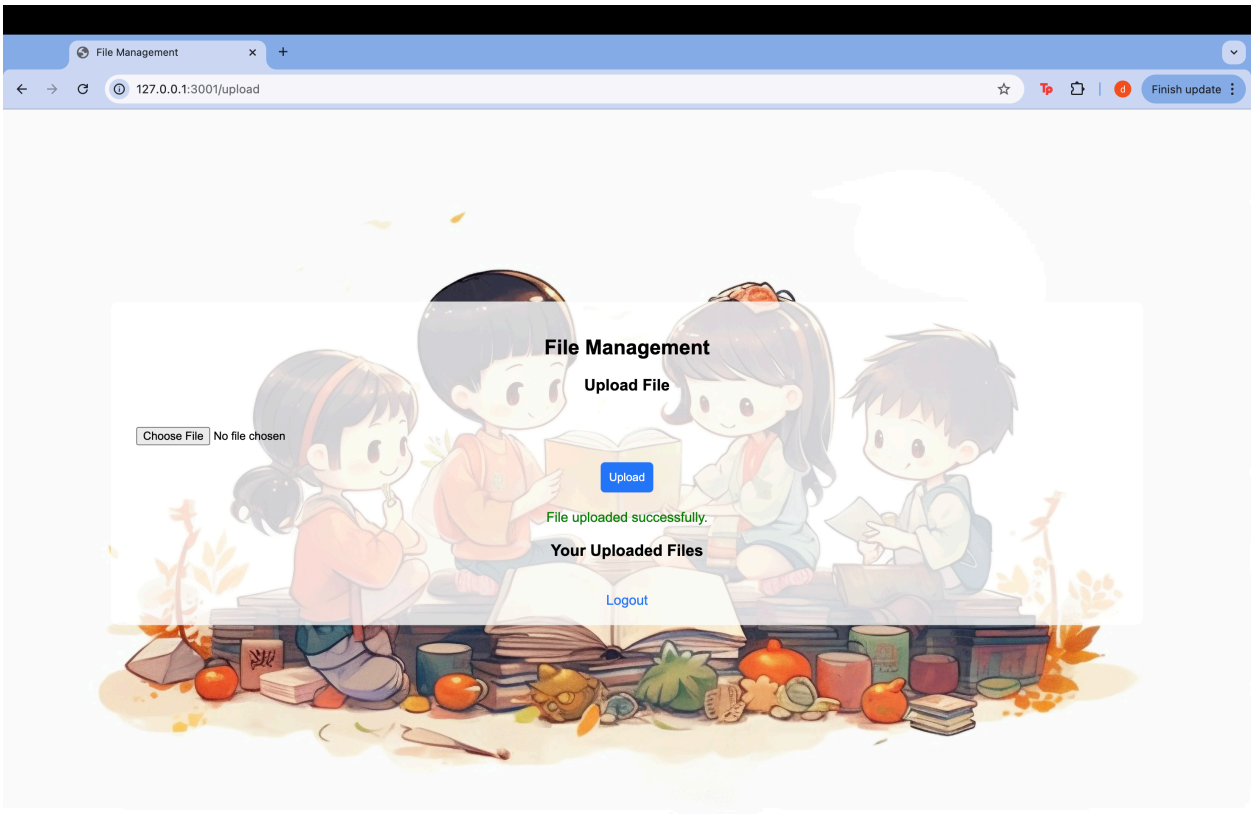
Step 22: Now again use the IP address and test the webserver.


Login
127.0.0.1:3001/login
Finish update
Login
Username
Password
Login
Don't have an account? Create one

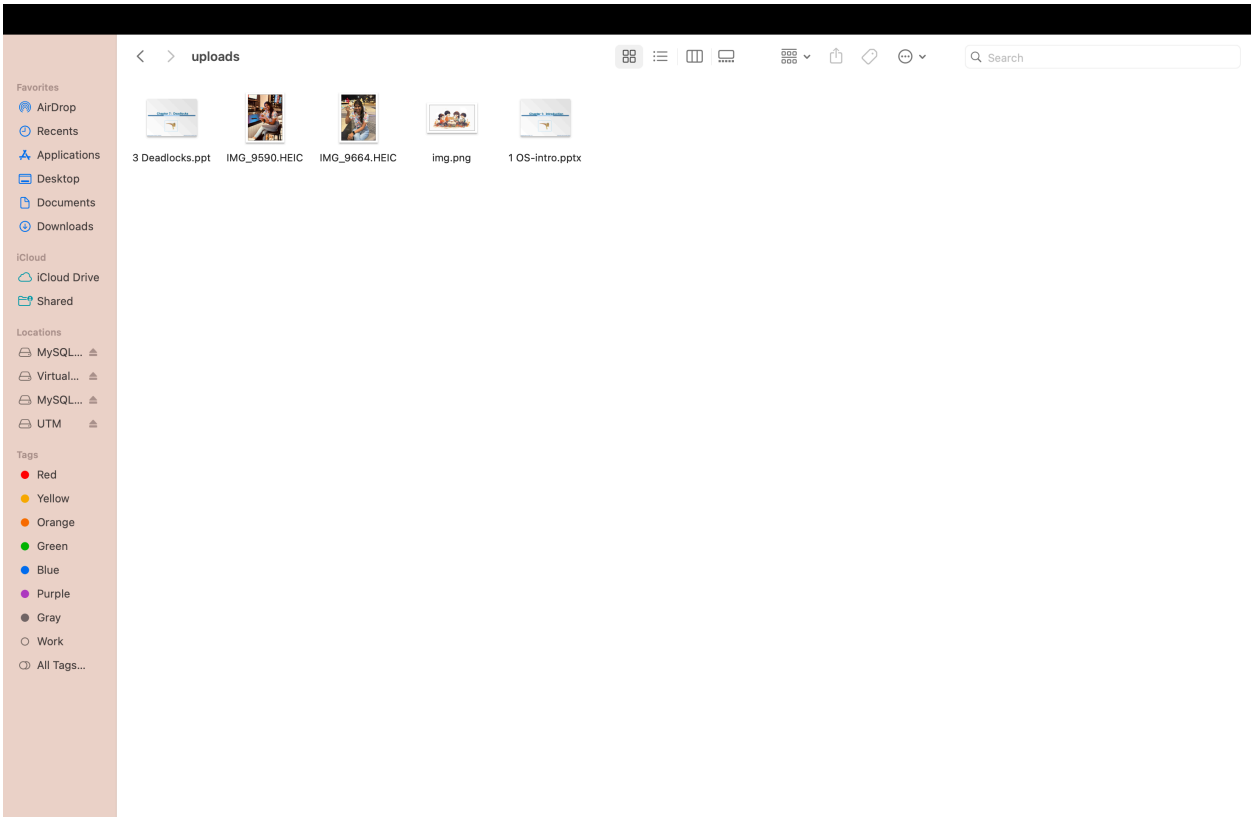

Login
127.0.0.1:3001/login
Finish update
Login
Dharani
Login
Don't have an account? Create one


Register
127.0.0.1:3001/create_account
Finish update
Register
Full Name
Email
Username
Password
Confirm Password
Register

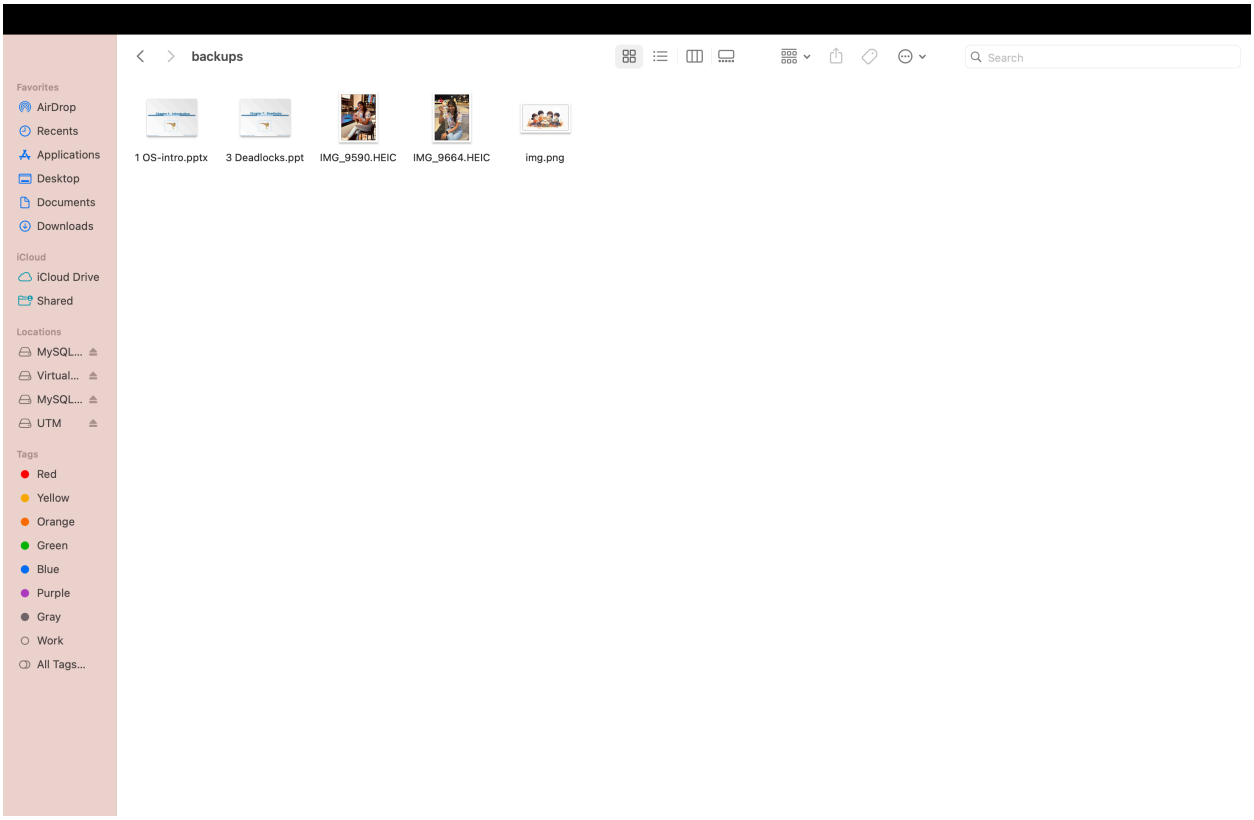

File Management
127.0.0.1:3001
Finish update
Choose File
No file chosen
Favorites
Recents
Applications
Desktop
Documents
Downloads
iCloud
iCloud Drive
Shared
Locations
MySQL Workbench co...
VirtualBox
MySQL Workbench co...
UTM
Tags
Downloads
Search
Previous 7 Days
Visual Studio Code
VSCode-darw...iversal (1).zip
Previous 30 Days
_Application 2024 (1).docx
_Application 2024.docx
1 OS-intro.pptx
2 process-thread.ppt
3 Deadlocks.ppt
4 OS-scheduling-rtos.ppt
5 Process Sy...hronization.ppt
22631.2861.2...FRE_en-us.iso
1926931_ReleaseLetter_1.pdf
Chapter 1: Introduction
1 OS-intro.pptx
Office Open XML presentation - 8.4 MB
Information
Show More
Created
Saturday, October 12, 2024 at 4:43 PM
Cancel
Open



Uploads folder:



Backup folder:



Final codes:

Rename.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Rename File</title>
</head>
<body>
    <h2>Rename File</h2>
    <form action="{{ url_for('rename', filename=filename) }}" method="post">
        <input type="text" name="new_filename" placeholder="New Filename"
required><br>
        <button type="submit">Rename</button>
    </form>
    <br>
    <a href="{{ url_for('files') }}">Back to File Management</a>
</body>
```

```
</html>
```

Create_account.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Register</title>
    <style>
        body {
            background: url('/static/images/img1.jpg') no-repeat center center fixed;
            background-size: cover;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
            font-family: Arial, sans-serif;
        }
        .register-container {
            background-color: rgba(255, 255, 255, 0.8);
            padding: 20px;
            border-radius: 10px;
            text-align: center;
            box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
        }
        input {
            margin: 10px 0;
            padding: 10px;
            width: 100%;
        }
        button {
            padding: 10px;
            width: 100%;
            background-color: #007BFF;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
        button:hover {
            background-color: #0056b3;
        }
    </style>
</head>
<body>
    <div class="register-container">
        <h2>Register</h2>
        <form action="/register" method="post">
```

```
        <input type="text" name="full_name" placeholder="Full Name" required>
        <input type="email" name="email" placeholder="Email" required>
        <input type="text" name="username" placeholder="Username" required>
        <input type="password" name="password" placeholder="Password" required>
        <input type="password" name="confirm_password" placeholder="Confirm Password" required>
        <button type="submit">Register</button>
      </form>
  </div>
</body>
</html>
```

setup_users.py

```
from werkzeug.security import generate_password_hash
import pymysql

# MySQL connection
db = pymysql.connect(host='localhost', user='root', password='Dharani@1108', database='nas_db')
cursor = db.cursor()

# Example user data
users = [
    {'username': 'Dharani', 'password': 'Dharani@1108', 'role': 'admin'},
    {'username': 'Jahnavi', 'password': 'janu123', 'role': 'user'},
    {'username': 'Keerthana', 'password': 'abc@111', 'role': 'user'},
    {'username': 'Sravani', 'password': 'choici$907', 'role': 'user'}
]

# Insert users into the database
for user in users:
    hashed_password = generate_password_hash(user['password'])
    cursor.execute("INSERT INTO users (username, password, role) VALUES (%s, %s, %s)",
                   (user['username'], hashed_password, user['role']))

db.commit()
print("Users added successfully.")
```

login.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
    <style>
        body {
            background: url('/static/images/img1.jpg') no-repeat center center fixed;
            background-size: cover;
            font-family: Arial, sans-serif;
            height: 100vh;
```

```
            margin: 0;
        }

        .login-container {
            background-color: rgba(255, 255, 255, 0.8);
            border-radius: 10px;
            padding: 20px;
            width: 300px;
            margin: auto;
            position: absolute;
            top: 50%;
            left: 50%;
            transform: translate(-50%, -50%);
            text-align: center;
        }

        .login-container input {
            margin: 10px 0;
            padding: 10px;
            width: 100%;
        }

        .login-container button {
            padding: 10px;
            width: 100%;
            background: #2575fc;
            color: white;
            border: none;
            border-radius: 5px;
        }

        .login-container a {
            color: #2575fc;
            text-decoration: none;
        }

        .login-container a:hover {
            text-decoration: underline;
        }
    </style>
</head>
<body>
    <div class="login-container">
        <h2>Login</h2>
        <form action="/login" method="post">
            <input type="text" name="username" placeholder="Username" required><br>
            <input type="password" name="password" placeholder="Password" required><br>
            <button type="submit">Login</button>
        </form>
        <p>Don't have an account? <a href="/create_account">Create one</a></p>
    </div>
```

```
</body>
</html>
```

f.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Upload Page</title>
    <style>
        body {
            background: url('/static/images/img.png') no-repeat center center fixed;
            background-size: cover;
            font-family: Arial, sans-serif;
            height: 100vh;
            margin: 0;
        }

        .upload-container {
            background-color: rgba(255, 255, 255, 0.8);
            border-radius: 10px;
            padding: 20px;
            width: 300px;
            margin: auto;
            position: absolute;
            top: 50%;
            left: 50%;
            transform: translate(-50%, -50%);
            text-align: center;
        }

        .upload-container input {
            margin: 10px 0;
            padding: 10px;
            width: 100%;
        }

        .upload-container button {
            padding: 10px;
            width: 100%;
            background: #2575fc;
            color: white;
            border: none;
            border-radius: 5px;
        }

        .upload-container a {
            color: #2575fc;
            text-decoration: none;
```

```
        }

        .upload-container a:hover {
            text-decoration: underline;
        }

        .success-message {
            color: green;
            font-size: 16px;
            margin-top: 20px;
        }
    </style>
</head>
<body>
    <div class="upload-container">
        <h2>Upload File</h2>
        <form action="/upload" method="post" enctype="multipart/form-data">
            <input type="file" name="file" required><br>
            <button type="submit">Upload</button>
        </form>
        {% if message %}
            <div class="success-message">{{ message }}</div>
        {% endif %}
        <br>
        <a href="/logout">Logout</a>
    </div>
</body>
</html>
```

Login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
    <style>
        body {
            background: url('/static/images/img1.jpg') no-repeat center center fixed;
            background-size: cover;
            font-family: Arial, sans-serif;
            height: 100vh;
            margin: 0;
        }

        .login-container {
            background-color: rgba(255, 255, 255, 0.8);
            border-radius: 10px;
            padding: 20px;
            width: 300px;
```

```
            margin: auto;
            position: absolute;
            top: 50%;
            left: 50%;
            transform: translate(-50%, -50%);
            text-align: center;
        }

        .login-container input {
            margin: 10px 0;
            padding: 10px;
            width: 100%;
        }

        .login-container button {
            padding: 10px;
            width: 100%;
            background: #2575fc;
            color: white;
            border: none;
            border-radius: 5px;
        }

        .login-container a {
            color: #2575fc;
            text-decoration: none;
        }

        .login-container a:hover {
            text-decoration: underline;
        }
    </style>
</head>
<body>
    <div class="login-container">
        <h2>Login</h2>
        <form action="/login" method="post">
            <input type="text" name="username" placeholder="Username" required><br>
            <input type="password" name="password" placeholder="Password" required><br>
            <button type="submit">Login</button>
        </form>
        <p>Don't have an account? <a href="/create_account">Create one</a></p>
    </div>
</body>
</html>
```

f.html

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>File Management</title>
    <style>
        body {
            background: url('/static/images/img.png') no-repeat center center fixed;
            background-size: cover;
            font-family: Arial, sans-serif;
            height: 100vh;
            margin: 0;
        }

        .container {
            background-color: rgba(255, 255, 255, 0.8);
            border-radius: 10px;
            padding: 20px;
            width: 80%;
            margin: auto;
            position: absolute;
            top: 50%;
            left: 50%;
            transform: translate(-50%, -50%);
            text-align: center;
        }

        .container input {
            margin: 10px 0;
            padding: 10px;
            width: 100%;
        }

        .container button {
            padding: 10px;
            background: #2575fc;
            color: white;
            border: none;
            border-radius: 5px;
        }

        .container a {
            color: #2575fc;
            text-decoration: none;
        }

        .container a:hover {
            text-decoration: underline;
        }

        .file-list {
            margin-top: 20px;
```

```
        }

        .file-item {
            margin: 10px 0;
        }

        .file-item button {
            margin-left: 10px;
        }

        .success-message {
            color: green;
            font-size: 16px;
            margin-top: 20px;
        }

        .error-message {
            color: red;
            font-size: 16px;
            margin-top: 20px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>File Management</h2>

        <!-- Upload Form -->
        <h3>Upload File</h3>
        <form action="/upload" method="post" enctype="multipart/form-data">
            <input type="file" name="file" required><br>
            <button type="submit">Upload</button>
        </form>

        <!-- Display Messages -->
        {% if message %}
            <div class="success-message">{{ message }}</div>
        {% endif %}
        {% if error_message %}
            <div class="error-message">{{ error_message }}</div>
        {% endif %}

        <!-- Uploaded Files List -->
        <h3>Your Uploaded Files</h3>
        <div class="file-list">
            {% for file in files %}
                <div class="file-item">
                    <span>{{ file['file_name'] }}</span>
                    <a href="{{ url_for('download', filename=file['file_name']) }}">Download</a> |
                    <a href="{{ url_for('rename', filename=file['file_name']) }}">Rename</a> |
```

```
                <form action="{{ url_for('delete', filename=file['file_name']) }}" method="post"
style="display:inline;">
                    <button type="submit">Delete</button>
                </form>
            </div>
        {% endfor %}
    </div>

    <br>
    <a href="/logout">Logout</a>
  </div>
</body>
</html>
```

b.py

```
from flask import Flask, request, jsonify, render_template, redirect, url_for, session, send_from_directory
import os
import pymysql
from werkzeug.security import generate_password_hash, check_password_hash
from apscheduler.schedulers.background import BackgroundScheduler
import shutil

app = Flask(__name__)
app.secret_key = 'your_secret_key'

# MySQL connection
db = pymysql.connect(host='localhost', user='root', password='Dharani@1108', database='nas_db')
cursor = db.cursor()

# Scheduler for backups
scheduler = BackgroundScheduler()

# File paths
UPLOAD_FOLDER = 'uploads'
BACKUP_FOLDER = 'backups'

# Ensure the folders exist
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(BACKUP_FOLDER, exist_ok=True)

@app.route('/')
def home():
    if 'username' in session:
        return render_template('f.html', username=session['username'], role=session['role'])
    return redirect(url_for('login'))

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
```

```
        password = request.form['password']
        cursor.execute("SELECT id, username, password, role FROM users WHERE username = %s",
(username,))
        user = cursor.fetchone()
        if user and check_password_hash(user[2], password):
            session['username'] = user[1]
            session['role'] = user[3]
            return redirect(url_for('home'))
        return "Invalid credentials!", 401
    return render_template('login.html')

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))

@app.route('/upload', methods=['POST'])
def upload():
    if 'username' not in session:
        return "Unauthorized", 401
    file = request.files['file']
    if file:
        filepath = os.path.join(UPLOAD_FOLDER, file.filename)
        file.save(filepath)
        cursor.execute("INSERT INTO files (file_name, file_path, uploaded_by) VALUES (%s, %s, %s)",
                       (file.filename, filepath, session['username']))
        db.commit()

        # Display success message
        message = 'File uploaded successfully.'
        return render_template('f.html', username=session['username'], role=session['role'],
message=message)
    return jsonify({'status': 'error', 'message': 'File upload failed.'})

@app.route('/backup', methods=['POST'])
def backup():
    if 'role' in session and session['role'] != 'admin':
        return "Permission denied!", 403
    try:
        # Create a backup by zipping the upload folder
        backup_filename = os.path.join(BACKUP_FOLDER, 'backup')
        shutil.make_archive(backup_filename, 'zip', UPLOAD_FOLDER)
        return jsonify({'status': 'success', 'message': 'Backup created successfully.'})
    except Exception as e:
        return jsonify({'status': 'error', 'message': f'Backup failed: {str(e)}'})

@app.route('/monitor')
def monitor():
    disk_usage = shutil.disk_usage("/")
    cpu_usage = os.popen("top -bn1 | grep 'Cpu(s)'").read()
    return jsonify({
```

```
        'disk_total': disk_usage.total,
        'disk_used': disk_usage.used,
        'cpu_usage': cpu_usage
    })

@app.route('/register', methods=['POST'])
def register():
    full_name = request.form['full_name']
    email = request.form['email']
    username = request.form['username']
    password = request.form['password']
    confirm_password = request.form['confirm_password']

    if password != confirm_password:
        return "Passwords do not match!", 400

    hashed_password = generate_password_hash(password)
    cursor.execute("INSERT INTO users (full_name, email, username, password) VALUES (%s, %s, %s,
%s)",
                   (full_name, email, username, hashed_password))
    db.commit()
    return redirect(url_for('login'))

def scheduled_backup():
    print("Scheduled backup running...")
    shutil.make_archive(os.path.join(BACKUP_FOLDER, 'auto_backup'), 'zip', UPLOAD_FOLDER)

scheduler.add_job(scheduled_backup, 'interval', hours=24)
scheduler.start()

@app.route('/create_account')
def create_account():
    return render_template('create_account.html')

@app.route('/files')
def files():
    if 'username' not in session:
        return redirect(url_for('login'))

    cursor.execute("SELECT file_name FROM files WHERE uploaded_by = %s", (session['username'],))
    files = cursor.fetchall()
    return render_template('f.html', files=files)

@app.route('/download/<filename>')
def download(filename):
    if 'username' not in session:
        return redirect(url_for('login'))

    file_path = os.path.join(UPLOAD_FOLDER, filename)
    if os.path.exists(file_path):
        return send_from_directory(UPLOAD_FOLDER, filename, as_attachment=True)
```

```
    return "File not found", 404

@app.route('/delete/<filename>', methods=['POST'])
def delete(filename):
    if 'username' not in session:
        return redirect(url_for('login'))

    cursor.execute("SELECT file_path FROM files WHERE file_name = %s AND uploaded_by = %s",
(filename, session['username']))
    file = cursor.fetchone()

    if file:
        file_path = file[0]
        if os.path.exists(file_path):
            os.remove(file_path)
            cursor.execute("DELETE FROM files WHERE file_name = %s", (filename,))
            db.commit()
            return redirect(url_for('files'))
    return "File not found", 404

@app.route('/rename/<filename>', methods=['GET', 'POST'])
def rename(filename):
    if 'username' not in session:
        return redirect(url_for('login'))

    if request.method == 'POST':
        new_filename = request.form['new_filename']
        cursor.execute("UPDATE files SET file_name = %s WHERE file_name = %s AND uploaded_by =
%s",
                       (new_filename, filename, session['username']))
        db.commit()
        return redirect(url_for('files'))

    return render_template('rename.html', filename=filename)

if __name__ == '__main__':
    app.run(debug=True, port=3001)
```

The Linux Web-Server for NAS Management project has been successfully completed, providing a simple yet powerful web interface to manage a Network Attached Storage (NAS) server.
The goal was to create a user-friendly system where users can remotely manage files, control user access, monitor system performance, and handle backups—all through a web browser.

- **Server Setup & Security**: I have set up a Linux server with Apache, MySQL, and Python/PHP, ensuring it was secure with proper firewalls and user access controls.

- **Web Interface**: A clean, easy-to-use web interface was created using HTML, CSS, and JavaScript, allowing users to interact with the NAS remotely. It connects to a MySQL database to store user info and file data. I was able to upload, download, delete and rename files on NAS.

- **Remote Access**: By setting up port forwarding, i enabled remote access to the NAS from anywhere, making it more convenient for users to interact with their files on the go.

- **User Access & Permissions**: Different user roles were created, each with specific permissions (read, write, admin), and secure login methods were added to protect sensitive data.

- **Backup & Restore**: I added features for automatic backups and restoring files, ensuring data is safe even in case of accidents or system failure.

This project now provides a fully functional, secure, and accessible way to manage NAS servers. It lays the groundwork for future improvements and is a valuable tool for anyone looking to simplify and secure their file management.